

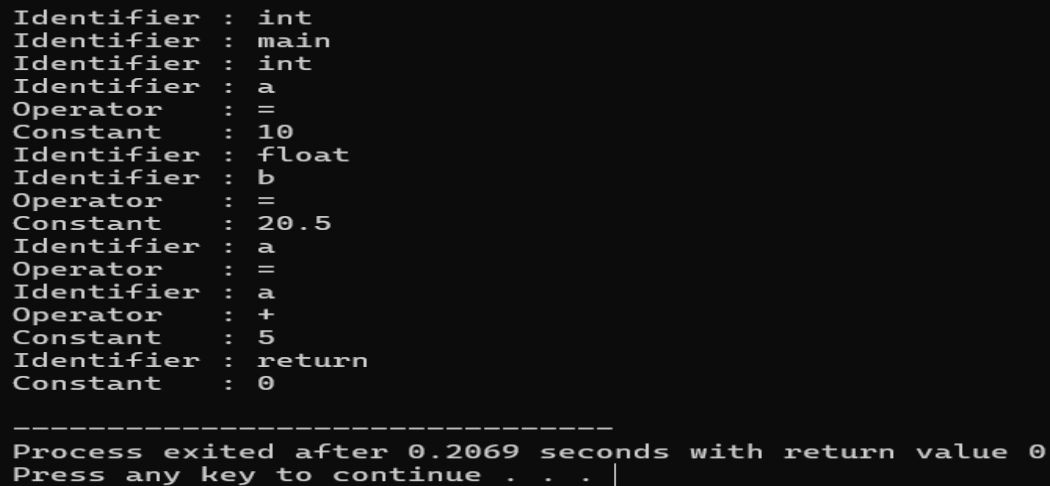
LAB EXPERIMENT 1 - 10

Q.1

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int isOperator(char ch)
{
    char operators[] = "+-*/%=<>!&|^";
    for (int i = 0; operators[i] != '\0'; i++)
    {
        if (ch == operators[i])
            return 1;
    }
    return 0;
}
int main()
{
    FILE *fp;
    char ch, token[50];
    int i;
    fp = fopen("input.c", "r");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 0;
    }
    while ((ch = fgetc(fp)) != EOF)
    {
        if (isspace(ch))
            continue;
        if (ch == '/')
        {
            char next = fgetc(fp);
            if (next == '/')
            {
                continue;
            }
        }
        token[i] = ch;
        i++;
        if (isOperator(ch) || ch == '\n')
        {
            token[i] = '\0';
            printf("%s\n", token);
            i = 0;
        }
    }
}
```

Output:



```
C:\Users\chall\OneDrive\Docu
Identifier : int
Identifier : main
Identifier : int
Identifier : a
Operator   : =
Constant   : 10
Identifier : float
Identifier : b
Operator   : =
Constant   : 20.5
Identifier : a
Operator   : =
Identifier : a
Operator   : +
Constant   : 5
Identifier : return
Constant   : 0

-----
Process exited after 0.2069 seconds with return value 0
Press any key to continue . . .
```

Q.2

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[500];

    printf("Enter a line: ");
    fgets(str, sizeof(str), stdin);

    int len = strlen(str);

    // Check for single-line comment
    if (str[0] == '/' && str[1] == '/')
    {
        printf("It is a Single-Line Comment.\n");
    }
    // Check for multi-line comment
    else if (str[0] == '/' && str[1] == '*' &&
             str[len - 3] == '*' && str[len - 2] == '/')
    {
        printf("It is a Multi-Line Comment.\n");
    }
    else
    {
        printf("It is NOT a Comment.\n");
    }

    return 0;
}
```

Output:

```
Enter a line: //
It is a Single-Line Comment.

-----
Process exited after 4.159 seconds with return value 0
Press any key to continue . . . |
```

Q.3:

```
char *keywords[] = {
    "int", "float", "char", "double", "if", "else",
    "while", "for", "return", "void", "break", "continue"
};

int isKeyword(char str[])
{
    int n = sizeof(keywords) / sizeof(keywords[0]);
    for (int i = 0; i < n; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int isOperator(char ch)
{
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
            ch == '=' || ch == '<' || ch == '>' || ch == '%' ||
            ch == '!' || ch == '&');
}

int main()
{
    FILE *fp;
    char ch, token[100];
    int i;

    fp = fopen("input.c", "r");
}
```

```

if (fp == NULL)
{
    printf("File not found!\n");
    return 0;
}

while ((ch = fgetc(fp)) != EOF)
{
    // Ignore spaces, tabs, new lines
    if (isspace(ch))
        continue;

    // Ignore comments
    if (ch == '/')
    {
        char next = fgetc(fp);

        if (next == '/')
        {
            while ((ch = fgetc(fp)) != '\n' && ch != EOF);
            continue;
        }
        else if (next == '*')
        {
            while ((ch = fgetc(fp)) != EOF)
            {
                if (ch == '*')
                {
                    if ((ch = fgetc(fp)) == '/')
                        break;
                }
            }
        }
    }

    // Identifier or Keyword
    if (isalpha(ch) || ch == '_')
    {
        i = 0;
        token[i++] = ch;

        while ((ch = fgetc(fp)) != EOF &&
            (isalnum(ch) || ch == '_'))
        {
            token[i++] = ch;
        }

        token[i] = '\0';

        if (isKeyword(token))
            printf("Keyword      : %s\n", token);
        else
            printf("Identifier   : %s\n", token);

        if (ch != EOF)
            fseek(fp, -1, SEEK_CUR);
    }

    // Constant
    else if (isdigit(ch))
    {
        i = 0;
        token[i++] = ch;

        while ((ch = fgetc(fp)) != EOF &&

```

Output:

```

Keyword      : int
Identifier   : main
Keyword      : int
Identifier   : a
Operator     : =
Constant     : 10
Keyword      : float
Identifier   : b
Operator     : =
Constant     : 20.5
Identifier   : a
Operator     : =
Identifier   : a
Operator     : +
Constant     : 5
Keyword      : return
Constant     : 0

-----
Process exited after 1.144 seconds with return value 0
Press any key to continue . . . |

```

Q.4:

```
int main()
{
    char op;

    printf("Enter an operator (+, -, *, /): ");
    scanf("%c", &op);

    switch(op)
    {
        case '+':
            printf("'+' is an Addition Operator.\n", op);
            break;

        case '-':
            printf("'-' is a Subtraction Operator.\n", op);
            break;

        case '*':
            printf("'*' is a Multiplication Operator.\n", op);
            break;

        case '/':
            printf("'/' is a Division Operator.\n", op);
            break;

        default:
            printf("'%' is NOT a Valid Arithmetic Operator.\n", op);
    }

    return 0;
}
```

Output:

```
Enter an operator (+, -, *, /): +
'+' is an Addition Operator.

-----
Process exited after 11.11 seconds with return value 0
Press any key to continue . . . |
```

Q.5

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch;
    int spaces = 0, tabs = 0, newlines = 0;

    fp = fopen("input.txt", "r");

    if (fp == NULL)
    {
        printf("File not found!\n");
        return 0;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch == ' ')
            spaces++;
        else if (ch == '\t')
            tabs++;
        else if (ch == '\n')
            newlines++;
    }

    fclose(fp);

    printf("Number of Spaces : %d\n", spaces);
    printf("Number of Tabs : %d\n", tabs);
    printf("Number of Newlines : %d\n", newlines);
}
```

Output:

```
File not found!

-----
Process exited after 2.409 seconds with return value 0
Press any key to continue . . . |
```

Q.6

```
#include <ctype.h>
#include <string.h>

int main()
{
    char id[100];
    int i;

    printf("Enter an identifier: ");
    scanf("%s", id);

    // First character must be a letter or underscore
    if (!(isalpha(id[0]) || id[0] == '_'))
    {
        printf("Invalid Identifier\n");
        return 0;
    }

    // Remaining characters must be letters, digits, or underscore
    for (i = 1; i < strlen(id); i++)
    {
        if (!(isalnum(id[i]) || id[i] == '_'))
        {
            printf("Invalid Identifier\n");
            return 0;
        }
    }

    printf("Valid Identifier\n");
    return 0;
}
```

Output:

```
Enter an identifier: total_marks
Valid Identifier

-----
Process exited after 23.31 seconds with return value 0
Press any key to continue . . . |
```

Q.7

```
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> e\n");
    printf("B -> e\n\n");

    printf("FIRST(S) = { a, b }\n");
    printf("FIRST(A) = { e }\n");
    printf("FIRST(B) = { e }\n");

    return 0;
}
```

Output:

```
Grammar:
S -> AaAb | BbBa
A -> e
B -> e

FIRST(S) = { a, b }
FIRST(A) = { e }
FIRST(B) = { e }

-----
Process exited after 1.605 seconds with return value 0
Press any key to continue . . . |
```

Q.8

```
#include <stdio.h>

int main()
{
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> e\n");
    printf("B -> e\n\n");

    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");

    return 0;
}
```

Output:

```
Grammar:
S -> AaAb | BbBa
A -> e
B -> e

FOLLOW(S) = { $ }
FOLLOW(A) = { a, b }
FOLLOW(B) = { a, b }

-----
Process exited after 1.906 seconds with return value 0
Press any key to continue . . . |
```

Q.9

```
#include <stdio.h>

int main()
{
    printf("Given Grammar:\n");
    printf("S -> (L) | a\n");
    printf("L -> L,S | S\n");

    printf("\nGrammar after eliminating Left Recursion:\n");
    printf("S -> (L) | a\n");
    printf("L -> SL'\n");
    printf("L' -> ,SL' | e\n");

    return 0;
}
```

Output:

```
Given Grammar:
S -> (L) | a
L -> L,S | S

Grammar after eliminating Left Recursion:
S -> (L) | a
L -> SL'
L' -> ,SL' | e

-----
Process exited after 2.324 seconds with return value 0
Press any key to continue . . . |
```

Q.10

```
#include <stdio.h>

int main()
{
    printf("Given Grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n");
    printf("E -> b\n");

    printf("\nGrammar after Left Factoring:\n");
    printf("S -> iEtSS' | a\n");
    printf("S' -> eS | e\n");
    printf("E -> b\n");

    return 0;
}
```

Output:

```
Given Grammar:
S -> iEtS | iEtSeS | a
E -> b

Grammar after Left Factoring:
S -> iEtSS' | a
S' -> eS | e
E -> b

-----
Process exited after 2.699 seconds with return value 0
Press any key to continue . . . |
```